

2025/2026

TORUŃ

AURUMVEIL

„AN OPTIMIZATION SYSTEM FOR RESOURCE
ALLOCATION AND ROUTING IN A DWARF KINGDOM”

Filip Bąk, Kacper Dębski, Samuel Kępski

Algorytmiczna Gildia Krasnoludków

„Algorytmy i struktury danych II”

Grupa: LA (1)

8 MAJ 2026

UMK

NASZ ZESPÓŁ

KACPER DĘBSKI

Odpowiedzialny za implementację wybranych modułów systemu, weryfikację koncepcji projektowych, przygotowanie wykresów dokumentacyjnych oraz współtworzenie prezentacji projektowej.

SAMUEL KĘPSKI

Odpowiedzialny za koordynację prac projektowych, opracowanie dokumentacji, przygotowanie materiałów graficznych oraz testowanie rozwiązania.

FILIP BĄK

Odpowiedzialny za implementację wybranych modułów systemu, przygotowanie elementów graficznych, wykresów dokumentacyjnych oraz współtworzenie prezentacji projektowej.

TECHNOLOGIE



Python



C++ 20



CMake Tools

Identyfikacja kluczowych problemów

OPTYMALNY PRZYDZIAŁ PRACY KRASNOLUDKÓW

Problem polega na wyznaczeniu **optymalnego przydziału krasnoludków do kopalń**, maksymalizującego wykorzystanie dostępnych stanowisk przy uwzględnieniu specjalizacji pracowników i ograniczeń pojemnościowych.

MINIMALIZACJA KOSZTU TRANSPORTU

Problem polega na wyznaczeniu przydziału krasnoludków do kopalń **minimalizującego całkowity koszt transportu**, przy jednoczesnym zachowaniu optymalnego wykorzystania dostępnych zasobów.

WYZNACZENIE GRANIC KRÓLESTWA

Problem polega na wyznaczeniu **zewnętrznych granic królestwa** obejmujących wszystkie aktywne kopalnie w celu efektywnego planowania patroli, monitorowania obszaru oraz zapewnienia ochrony.

WYZNACZANIE DEKAMETROWCA GRANICY

Problem polega na identyfikacji **Dekametrowca wydającego najgłośniejsze rozkazy** na wskazanym odcinku granicy, bez konieczności analizowania wszystkich krasnoludków w królestwie.

ZACHOWANIE I WYSZUKIWANIE WIEDZY KRÓLESTWA

Problem polega na efektywnym **przechowywaniu wiedzy królestwa oraz szybkim ich wyszukiwaniu**, przy jednoczesnej optymalizacji wykorzystania pamięci i zapewnieniu sprawnego dostępu do danych.

Model problemu i zastosowane rozwiązanie

Wyzwanie Królestwa:

Królestwo musi obsadzić **wszystkie dostępne stanowiska w kopalniach**. Każdy krasnolud posiada określone umiejętności, a liczba miejsc pracy jest ograniczona. Należy znaleźć **przydział maksymalizujący wykorzystanie dostępnych zasobów** przy zachowaniu wszystkich ograniczeń.

Idea działania:

- **Utworzenie sieci przepływowej** reprezentującej możliwe przydziały krasnoludków do kopalń.
- **Wyznaczenie ścieżki powiększającej** umożliwiającej zwiększenie przepływu w sieci.
- **Zwiększenie liczby przydzielonych krasnoludków** zgodnie z dostępną przepustowością sieci.
- **Powtarzanie procesu** do momentu, gdy nie będzie możliwe dalsze zwiększanie przepływu.
- **Uzyskanie optymalnego przydziału krasnoludków** maksymalizującego wykorzystanie dostępnych miejsc pracy.

Dlaczego Algorytm Edmondsa-Karpa?

Problem przydziału krasnoludków można przedstawić jako **sieć przepływową**, w której przepływ odpowiada liczbie przydzielonych pracowników, a ograniczenia pojemności odzwierciedlają liczbę miejsc pracy oraz wymagane umiejętności. **Maksymalny przepływ** wyznacza największą liczbę poprawnych przydziałów do kopalń.

Model problemu i zastosowane rozwiązanie

Wyzwanie Królestwa:

Krasnoludki codziennie przemieszczają się z domków do przydzielonych kopalń. Ze względu na **różne odległości między lokalizacjami** królestwo dąży do wyznaczenia **tras minimalizujących łączny koszt transportu** przy zachowaniu **sprawnej organizacji pracy**.

Idea działania:

- **Umieszczenie wierzchołka startowego w kolejce** i nadanie mu odległości równej zero.
- **Pobieranie kolejnych wierzchołków z kolejki** oraz relaksacja wychodzących z nich krawędzi.
- **Aktualizacja odległości do sąsiednich wierzchołków** po znalezieniu krótszej ścieżki.
- **Ponowne dodawanie wierzchołków do kolejki** w przypadku poprawy ich odległości.
- **Powtarzanie procesu** do momentu opróżnienia kolejki.
- **Wyznaczenie tras o minimalnym koszcie transportu** pomiędzy domkami krasnoludków a kopalniami.

Dlaczego Algorytm 'Esopo-Pape?

Problem transportu w królestwie można sprowadzić do wyznaczania **najkrótszych ścieżek** w grafie reprezentującym połączenia między domkami krasnoludków a kopalniami. Algorytm **d'Esopo-Pape** został wybrany ze względu na **wysoką efektywność praktyczną** oraz możliwość szybkiego wyznaczania **tras minimalizujących całkowity koszt transportu**.



Model problemu i zastosowane rozwiązanie

Wyzwanie Królestwa:

Aktywne kopalnie rozmieszczone są w różnych częściach królestwa. Aby usprawnić **planowanie patroli** oraz **wyznaczyć trasę codziennego obchodu księcia**, konieczne jest **określenie zewnętrznych granic obejmujących wszystkie kopalnie**.

Idea działania:

- **Wybór punktu początkowego** o najmniejszej współrzędnej y.
- **Posortowanie pozostałych punktów** względem kąta utworzonego z punktem początkowym.
- **Utworzenie początkowej otoczki** z pierwszych punktów uporządkowanej listy.
- **Analiza kolejnych punktów** i sprawdzanie kierunku skrętu.
- **Usuwanie punktów powodujących skręt** do wnętrza otoczki oraz dodawanie nowych punktów do jej granicy.
- **Wyznaczenie zewnętrznej granicy obejmującej wszystkie kopalnie** na podstawie otoczki wypukłej.

Dlaczego Algorytm Grahama?

Problem wyznaczenia granic królestwa można sprowadzić do wyznaczenia **otoczki wypukłej** dla zbioru punktów reprezentujących aktywne kopalnie. Algorytm Grahama umożliwia efektywne wyznaczenie **zewnętrznej granicy obejmującej wszystkie lokalizacje**, nawet dla większej liczby punktów.

Model problemu i zastosowane rozwiązanie

Wyzwanie Królestwa:

Straż graniczna regularnie monitoruje wybrane odcinki granicy. Aby szybko reagować na zagrożenia, konieczne jest wskazanie **dekametrowca wydającego najgłośniejsze rozkazy** na analizowanym fragmencie granicy, bez sprawdzania **wszystkich strażników**.

Idea działania:

- **Podział granicy na mniejsze przedziały** i utworzenie struktury drzewa.
- **Przypisanie poziomów głośności dekametrowcom** reprezentowanym przez liście drzewa.
- **Wyznaczenie wartości węzłów wewnętrznych** na podstawie wartości ich dzieci.
- **Rozbicie zapytania na odpowiednie przedziały** reprezentowane przez węzły drzewa.
- **Połączenie wyników z wybranych przedziałów** w celu uzyskania odpowiedzi.
- **Wskazanie najgłośniejszego dekametrowca** na analizowanym odcinku granicy.

Dlaczego drzewo przedziałowe?

Problem można sprowadzić do wielokrotnego wyszukiwania **maksymalnej wartości na wybranym odcinka granicy**. Drzewo przedziałowe umożliwia **szybkie odpowiadanie na zapytania przedziałowe** oraz **efektywną aktualizację danych**, bez konieczności przeglądania wszystkich dekametrowców.

Model problemu i zastosowane rozwiązanie

Wyzwanie Królestwa:

Królestwo gromadzi **ogromne ilości informacji** dotyczących kopalń, mieszkańców oraz funkcjonowania królestwa. Konieczne jest **ograniczenie rozmiaru przechowywanych książ**, przy jednoczesnym zachowaniu możliwości pełnego odtworzenia zapisanych danych.

Idea działania:

- **Zliczenie częstotliwości występowania symboli** w danych wejściowych.
- **Utworzenie kolejki priorytetowej** zawierającej wszystkie symbole wraz z ich częstotliwościami.
- **Połączenie dwóch najmniej częstych symboli** w nowy węzeł o wadze równej sumie ich częstotliwości.
- **Powtarzanie procesu łączenia**, aż do utworzenia jednego drzewa Huffmana.
- **Wygenerowanie kodów binarnych** poprzez przejście od korzenia do liści (0 – lewe dziecko, 1 – prawe dziecko).
- **Zakodowanie danych** przy użyciu wygenerowanych kodów, gdzie częstsze symbole otrzymują krótsze kody.

Dlaczego Algorytm Huffmana?

Problem przechowywania wiedzy sprowadza się do **zmniejszenia rozmiaru danych bez utraty informacji**. Algorytm Huffmana wykorzystuje **częstotliwość występowania symboli**, przypisując **krótsze kody częściej używanym znakom** JSONa, co pozwala skutecznie ograniczyć rozmiar elektronicznych książ przy zachowaniu możliwości ich **pełnego odtworzenia**.

Model problemu i zastosowane rozwiązanie

Wyzwanie Królestwa:

Królestwo gromadzi wyniki analiz oraz wiedzę dotyczącą kopalń, granic i mieszkańców w **elektronicznych księgach**. Wraz ze wzrostem liczby zapisów konieczne staje się **szybkie odnajdywanie wcześniej wyznaczonych rozwiązań** bez ponownego wykonywania kosztownych obliczeń.

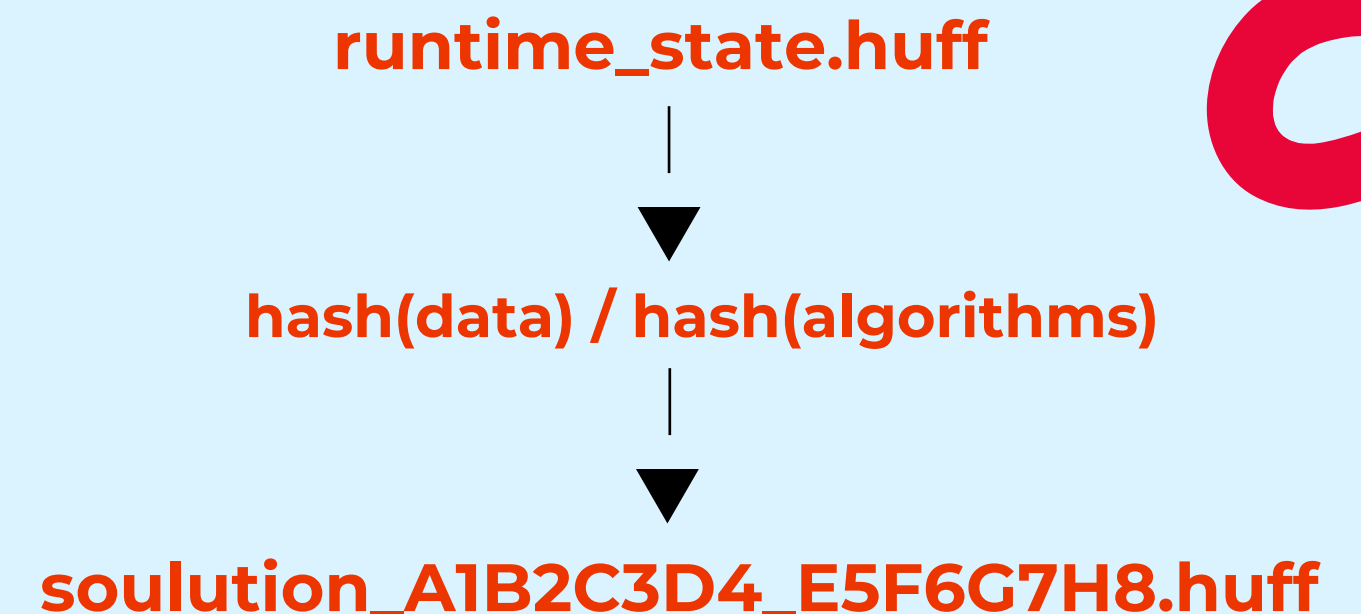
Idea działania:

- **Wygenerowanie identyfikatorów haszujących** na podstawie danych wejściowych i użytych algorytmów.
- **Utworzenie nazwy pliku rozwiązania** na podstawie wygenerowanych skrótów.
- **Sprawdzenie istnienia wcześniej zapisanego wyniku na dysku.**
- **Odczyt i dekompresja zapisanego rozwiązania** w przypadku znalezienia pliku.
- **Pominięcie ponownych obliczeń** dzięki wykorzystaniu wcześniej wyznaczonego rozwiązania.
- **Wyświetlenie odtworzonego rozwiązania użytkownikowi.**

Dlaczego Hash-based Lookup?

Wiele zapytań dotyczy danych, które zostały już wcześniej przetworzone. Technika **Hash-based Lookup** umożliwia identyfikację identycznych zestawów danych za pomocą **funkcji haszujących** i natychmiastowe odczytanie zapisanych wyników. Pozwala to znacząco skrócić czas wyszukiwania oraz ograniczyć liczbę zbędnych obliczeń.

Identyfikacja pliku:



2025/2026

TORUŃ

PYTANIA?

8 MAJ 2026

UMK